How Numerical and Variance-Reduction Techniques Affect the Accuracy and Efficiency of Monte Carlo Option Pricing

Oct 15, 2026 · @yash

Abstract

Monte Carlo option prices converged exactly as theory predicts and their reported error bars were honest, until fewer than about 50 simulated paths paid anything; then the error bars collapsed, and the classical speed-up techniques either left the failure in place or created it. This report asks: *how do numerical and variance-reduction techniques affect the accuracy and computational efficiency of Monte Carlo option pricing?*

I built a pricing engine in Python and C++, validated it against Black-Scholes and textbook binomial-tree values, and wrote a measurement harness that checks any estimator for bias, honest standard errors and 95% confidence-interval coverage over hundreds of independent seeds. Using it, I measured plain Monte Carlo, antithetic variates, control variates, their combination and importance sampling across 16 European options and a sweep of far out-of-the-money strikes.

Plain Monte Carlo's error fell like $1/\sqrt{N}$ (fitted slopes −0.490 to −0.501), and its reported standard error matched the true spread to within 4%. Coverage, however, depended on the number of paths that finished in the money: it fell from 0.95 to 0.40 when about one path paid, and to 0.03 when almost none did. Control variates cut variance by up to 111×, but where they worked best they reproduced this same failure (coverage 0.158). Only importance sampling, which changes where paths go, restored coverage, from 0.000 to 0.934 at the most extreme strike. On performance, single-threaded C++ matched vectorised NumPy; the 8.2× cost advantage came from parallelism. The answer is that variance reduction improves accuracy only where the estimator already sees the events that matter, so it must be judged by coverage, not by variance alone.

1. Introduction

An option gives its holder the right, but not the obligation, to buy or sell an asset at a fixed price (the *strike*) on or before a future date. Banks and trading firms price millions of options a day, and for simple options there is an exact formula: Black-Scholes. For most

real products there is not, so prices are estimated by simulation. Monte Carlo pricing simulates thousands or millions of possible futures for the asset, computes the option's payoff in each, and averages the results.

A Monte Carlo price always comes with an error bar, the *standard error*, which says how far the estimate is likely to be from the true price. Two things matter in practice. The first is cost: halving the error needs four times as many paths. The second is trust: a trader relies on the error bar to know whether the price is good enough. Many techniques exist to reduce the error for the same number of paths, but they are usually judged by how much they reduce variance, not by whether the error bar they report can still be trusted.

Research question: *How do numerical and variance-reduction techniques affect the accuracy and computational efficiency of Monte Carlo option pricing?*

To answer it, I built and validated a pricing engine, then measured each technique not only by how much it reduced the error, but by whether its 95% confidence intervals actually contained the true price 95% of the time, and by how much computing time it cost.

2. Background

Black-Scholes. The Black-Scholes model assumes the stock follows geometric Brownian motion: it grows on average at the risk-free rate r with constant volatility σ. Under this model a European call (exercisable only at expiry T) has an exact price:

```
C = S\,N(d_1) - K e^{-rT} N(d_2), \qquad d_{1,2} = \frac{\ln(S/K) + (r \pm \sigma^2/2)T}{\sigma\sqrt{T}}
```

Here N is the standard normal distribution. N(d2) is the risk-neutral probability that the option finishes in the money, so the formula reads as "what you receive minus what you pay". Because this price is exact, it is the benchmark against which every numerical method in this report is measured. The *Greeks* are its sensitivities: delta (to the stock price), gamma (how fast delta changes), vega (to volatility), theta (to time) and rho (to the interest rate).

Binomial trees. A binomial tree splits the option's life into N steps, and in each step the stock moves up by a factor u = e^(σ√Δt) or down by d = 1/u. The option is valued backwards from expiry: each node is the discounted, probability-weighted average of the two nodes after it. Trees can also price American options, which may be exercised early, by taking at each node the larger of continuing and exercising now.

Monte Carlo and its error bar. Monte Carlo simulates N terminal stock prices, averages the discounted payoffs, and reports the standard error s/√N, where s is the sample standard deviation of the payoffs. The 95% confidence interval is the estimate ± 1.96 standard errors. This interval relies on the central limit theorem: the average of many independent payoffs is approximately normally distributed. Its *coverage* is the fraction of runs in which the interval contains the true price, and it should be 0.95.

Variance reduction. Because the error falls only like 1/√N, gaining one extra decimal place costs 100× the paths. Variance-reduction techniques instead attack s:

Technique	Idea
Antithetic variates	For every random draw Z, also use −Z, and average each pair; errors in opposite directions partly cancel
Control variates	Use a related quantity with a known mean (here, the discounted terminal stock price) to correct the estimate
Importance sampling	Shift the simulation so more paths land where the payoff is non-zero, then reweight each path so the estimate stays unbiased

3. Building and validating the engine

Every pricing method was checked against a known answer before it was used in an experiment; the engine passes 2,950 automated tests.

Black-Scholes and the Greeks. The engine reproduces Hull's textbook example (S = 42, K = 40) exactly: d1 = 0.7693, d2 = 0.6278, call 4.76, put 0.81. Put-call parity holds across a 108-point grid of inputs. The Greeks match Hull's second example (S = 49, K = 50):

Greek	Engine	Hull
Delta	0.5216	0.522
Gamma	0.0655	0.066
Vega	12.105	12.1
Theta (per year)	−4.305	−4.31

Greek	Engine	Hull
Rho	8.907	8.91

Every Greek, for calls and puts, was also checked against a finite-difference estimate. Two of my own test assumptions were wrong and were corrected by measurement: at-the-money gamma grows like 1/√T, so a 1,000× shorter expiry raises it about 32×, not 100×.

Binomial tree. The Cox-Ross-Rubinstein tree reproduces Hull's 5-step American put (4.4885 against 4.49) and a two-step tree checked by hand. Its error against Black-Scholes falls like 1/N: error × N settled at 1.995 to 2.000 from N = 50 to 3,200, with a fitted log-log slope of −1.00. At the money, even step counts always undershoot and odd step counts always overshoot, with no exceptions from N = 10 to 5,001. Averaging an N and N+1 pair leaves only 7.1% of the better single error, a 14× accuracy gain for one extra tree.

American options. Two results confirm the theory of early exercise. An American call on a non-dividend stock was worth exactly the same as the European call in all 51 test cases, because exercising early only means paying the strike sooner. The American put was worth more (at r = 10%: +9.41 at S = 60, +1.06 at S = 100, +0.01 at S = 140), and at r = 0 that premium was exactly zero everywhere. This shows the early-exercise premium of a put is the interest earned by receiving the strike sooner.

4. Method

4.1 The measurement harness. All experiments ran through one harness that accepts any estimator as a function of (option, number of paths, seed). For each configuration it runs the estimator with many independent seeds and compares the results with the exact Black-Scholes price. Because every method passes through identical measurement code, differences between methods cannot come from differences in how they were measured. Before use, I calibrated the harness with fake estimators of known behaviour: an honest one, one that reports error bars 4× too small, an under-confident one, and a biased one. It correctly flagged each: for example, the over-confident estimator showed a reported-to-actual error ratio of 0.25 and coverage below 0.65.

4.2 Options tested. The base case is a European call with S = 100, K = 100, r = 5%, σ = 20% and T = 1 year (Black-Scholes price 10.45). The main grid contains 16 options across five levels of moneyness (deep out of the money to deep in the money) and three maturities (1

month, 1 year, 5 years). A separate sweep used six-month calls with strikes from 120 to 200, to find where Monte Carlo breaks down. A straddle (call plus put) was added in Phase 4 to test antithetic variates on a symmetric payoff.

4.3 Metrics. For each configuration I measured:

- **RMSE and bias** against the exact price, across 500 seeds (more for some checks);
- **Honesty ratio**: the mean reported standard error divided by the actual spread of the estimates (1.0 = honest);
- **Coverage**: the fraction of 95% intervals that contained the true price;
- **Variance reduction factor**: plain Monte Carlo's variance divided by the method's variance, at the same number of paths;
- **Cost**: variance × runtime, where lower is better.

4.4 Fair comparisons. Phase 3 showed that the time per path is not constant: it is high for small runs (fixed overhead) and rises again for large runs (memory traffic). Comparing methods at different path counts would therefore partly measure the memory system, so all variance-reduction comparisons use equal path counts, mostly N = 100,000. Timings were repeated in an interleaved order and the fastest run reported, because background noise can only slow a run down. With 500 seeds, a single coverage figure is only accurate to about ±0.02, so values between 0.93 and 0.97 are treated as consistent with 0.95.

4.5 Setup. Python with NumPy and SciPy; C++17 compiled with GCC (WinLibs) and loaded into Python as a C-ABI DLL through ctypes; an Intel i7-14650HX running Windows 11. All random numbers come from fixed seeds, so every result can be reproduced.

5. Results I: convergence and the error bar

Plain Monte Carlo behaved exactly as theory predicts for every option in the grid, and its error bar was honest, except when almost no paths paid anything.

5.1 Convergence. The fitted log-log slope of RMSE against N was −0.496 for the base case and between −0.490 and −0.501 for all 16 options, against a theoretical −0.5. Using 1,000× more paths improved accuracy 30.5× (theory: √1000 = 31.6). For the base-case call, RMSE fell from 0.4535 at N = 1,000 to 0.1516, 0.0468 and 0.0149 at N = 10^6. Bias was negligible: −0.0007 at N = 10^6 on a price of 10.45.

5.2 Honesty of the error bar. Across all 64 combinations of option and path count, the mean reported standard error divided by the actual spread of 500 estimates was between 0.966 and 1.038. The error bar plain Monte Carlo reports is therefore a reliable measure of

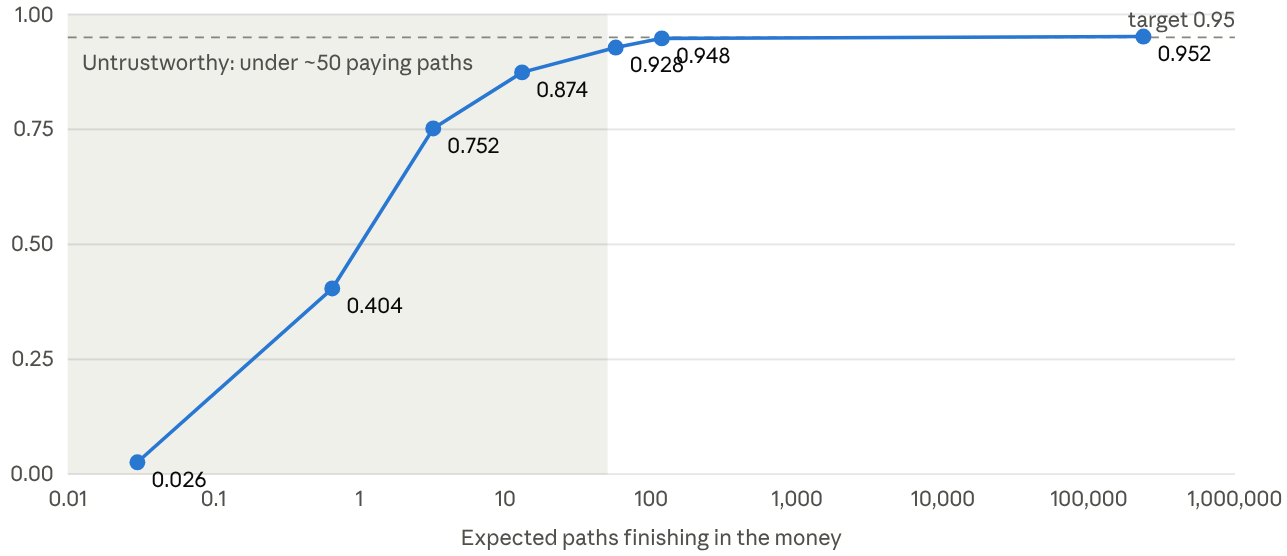
its real error over four orders of magnitude of N.

5.3 Cost of accuracy. A ±1 cent 95% error bar on the base-case call needs about 8.3 million paths (0.23 s). For a deep in-the-money 5-year call it needs about 130 million paths (3.6 s), because its payoff varies more.

5.4 Where coverage breaks. Coverage stayed between 0.93 and 0.96 across the whole 16-option grid, including a deep out-of-the-money call with payoff skewness of 8.71. I had predicted failure wherever |skewness|/√N exceeded 0.1; that prediction was wrong. The strike sweep found the real cause: the number of paths that finish in the money.

95% intervals only work once about 50 paths pay anything

Coverage of plain Monte Carlo 95% intervals vs expected number of in-the-money paths (log scale)



Phase 3 strike sweep (6-month calls, K = 120 to 200) and base grid · 500 seeds per point

With about 118 paying paths, coverage was 0.948. With about 13 it fell to 0.874, with about 3 to 0.752, with about one to 0.404, and with almost none to 0.026. The observed threshold in |skewness|/√N was about 0.3, three times the rule of thumb I had used.

5.5 The mechanism. Dividing each estimate's error by its own reported standard error should give a standard normal distribution. For the K = 160 call at N = 1,000, it instead gave a spike just below +1 with a long left tail (skewness −10.98). The spike is the case where exactly one path pays. With one payoff X among N paths, the estimate is about X/N and the standard error is also about X/N, so their ratio is pinned near 1 whatever the true price is. When no path pays, the estimator reports 0 ± 0 for a price that is not zero. In both cases, the estimator is most confident precisely when it has the least information.

6. Results II: variance reduction

Classical variance reduction cut variance by up to 521× on ordinary options but did nothing for the rare-event failure; importance sampling was the only method that fixed it.

6.1 Variance reduction factors. Each value is plain Monte Carlo's variance divided by the method's variance at N = 100,000 paths; above 1 is better.

Option	Antithetic	Control	Both	Importance sampling
ATM, 1 year	1.7×	6.2×	28×	0.7×
OTM, 5 years	1.4×	10.7×	44×	0.8×
Deep ITM, 1 year	9.3×	111×	521×	0.0×
K = 160, 6 months	0.9×	1.0×	0.9×	1,745×

Antithetic variates were also about 25% faster per path than plain Monte Carlo, because they need half as many random numbers. The antithetic standard error, computed over pair averages, matched the true spread to a ratio of 1.0002. Computed over individual paths it would have been wrong in both directions: too small for a call, whose paired payoffs are negatively correlated, and too large for a straddle, whose paired payoffs are positively correlated.

6.2 Combining methods. The combined factor is neither the sum nor the product of the two separate factors. It follows:

```
\text{both} = \frac{1}{(1+\rho_A)(1-\rho_{AB}^2)}
```

where ρ_A is the correlation between paired payoffs and ρ_AB is the payoff-control correlation recomputed on pair averages. Antithetic pairing removes the odd (antisymmetric) part of both payoff and control. Usually what is left correlates more tightly, so the combination beats the product: for the ATM call, ρ rose from 0.924 to 0.964 and the combination was 2.08× the product. But when the payoff is already almost an exact linear function of the control, pairing removes exactly what the control was exploiting: for the deep ITM call, ρ fell from 0.995 to 0.990 and the combination was 0.46× the product. The formula predicted the measured ratios to three significant figures.

6.3 Where each method fails.

- **Antithetic variates on a straddle.** The factor was 10.5× at K = 70, fell to 0.60× (worse than plain) at K = 100, and recovered to 3.2× at K = 130. The worst point brackets the median terminal price, 100 × e^(0.05 − 0.02) = 103.05, where a draw and its mirror give the most similar payoffs.
- **Importance sampling where it is not needed.** It increased variance at the money (0.7×) and far more in the money.
- **Control variates succeeding too well.** Deep in the money, the payoff is an exact linear function of the control except on paths that finish below the strike. Variance fell 14,000,000×, but 83.0% of runs never sampled a path below the strike, and exactly those runs reported a standard error below 10^−12. Coverage fell to 0.158, against 0.952 for plain Monte Carlo. Variance reduction recreated the failure from section 5.
- **Estimating the control coefficient on the same paths** introduces a bias that falls like 1/N: −0.147 at N = 50, −0.038 at N = 200 and −0.0058 at N = 1,000. Fitting the coefficient on a separate pilot sample removed it, at the cost of 7–35% more spread. By N = 10,000 the bias is under 1% of a standard deviation.

6.4 Fixing the rare-event failure. Coverage of 95% intervals at N = 1,000 for six-month calls:

Strike	Plain	Antithetic	Control	Both	Importance sampling
K = 140	0.876	0.886	0.876	0.878	0.962
K = 160	0.404	0.426	0.404	0.418	0.940
K = 180	0.026	0.024	0.026	0.024	0.942
K = 200	0.000	0.000	0.000	0.000	0.934

The classical methods left coverage where it was. Importance sampling, which shifts the paths towards the strike and reweights them, restored it in every case, including K = 200, where plain Monte Carlo never saw a single paying path.

7. Results III: performance

Single-threaded C++ was no faster than vectorised NumPy; the large gains came from leaving the Python interpreter and from using several CPU cores.

Throughput in millions of paths per second (fastest of interleaved repeats):

Paths	Python loop	NumPy	C++, 1 thread	C++, 24 threads
10^3	2.85	5.77	15.1	0.27
10^5	2.94	51.8	38.6	32.0
10^7	—	39.4	38.5	324.1

Where the speed-ups came from:

1. **Interpreter to NumPy: 15–20×.** A pure Python loop spends about 340 ns per path, roughly 1,000 CPU cycles for about 20 arithmetic operations. NumPy removes this by running the loop in compiled code.
2. **NumPy to single-threaded C++: none (0.98×).** NumPy's inner loops are already compiled and vectorised, so a scalar C++ loop has no arithmetic to win. Compiling with `-march=native` changed nothing, because the loop is dominated by exp, log, sin and cos calls that do not auto-vectorise.
3. **Memory.** The C++ estimators keep a running mean and variance instead of storing every payoff, so their cost stayed flat at 25.9 ns per path from 10^4 to 10^7 paths. NumPy's rose from 16.6 to 25.4 ns as its arrays outgrew the CPU cache. This explains the rising half of the U-shaped cost curve seen in Phase 3.
4. **Threads.** Speed-ups were 1.99×, 3.95× and 7.34× on 2, 4 and 8 threads, then flat: the CPU has 8 performance cores. For small runs, threads hurt: at 10^3 paths, starting 24 threads took about 3.7 ms, or 3,726 ns per path against 66 ns on one thread.

Cost. At 10^7 paths, 24-thread C++ cost 6.68 × 10^−7 in variance × runtime against 5.50 × 10^−6 for NumPy, 8.2× cheaper per unit of accuracy. This advantage comes from parallelism, not from the language.

Correctness and reproducibility. The C++ estimators ran through the same harness as the Python ones: unbiased over 300 seeds, with honest standard errors, correct antithetic pairing and the importance-sampling rescue reproduced. Each random draw is a pure function of (seed, path index), so threads cannot share random numbers, and the work is split into a fixed number of chunks whatever the thread count. As a result, 1, 2, 3, 4, 8 and 16 threads return a bitwise identical price and standard error.

Two measurement errors caught. Running all repeats of one implementation back to back inflated the C++ time by 1.9× (48.5 against 25.7 ns per path); interleaving the repeats fixed this. Benchmarking also exposed a waste in my Python code: every estimator recomputed the same normal quantile on each call, costing 27 µs, 12% of a 1,000-path run.

8. Discussion

Variance is not the same as accuracy. The most important result is that a smaller reported variance does not guarantee a trustworthy price. The deep in-the-money control variate cut variance 14,000,000×, yet its 95% intervals were right only 15.8% of the time. A comparison that ranked methods by variance alone would have named it the best method in the study. Coverage, the question of whether the error bar can be believed, is the measure that exposed it.

The failure has one cause: missing events. Every coverage failure in this project, for plain Monte Carlo far out of the money and for the control variate deep in the money, happened when the simulation rarely or never sampled the part of the distribution that decides the price. A standard error is computed from the paths the simulation saw, so it cannot warn about paths it never saw. That is why the estimator was most confident when it knew least, and why the number of paying paths predicted coverage better than skewness did.

Why only importance sampling fixed it. Antithetic and control variates reuse the same paths more cleverly; they cannot create information about a region no path reaches. Importance sampling changes where the paths go. That is why it gave a 1,745× variance reduction at K = 160, where the other methods gave about 1×, and also why it made things worse for ordinary options, where the shift moves paths away from where the payoff already was. No method was best everywhere; each one's benefit depended on the shape of the payoff.

Combining methods has a predictable result. The combined variance reduction followed a formula based on how pairing changes the payoff-control correlation. This turns a surprising table into something that can be predicted before running it, and explains why combining helped the ATM call but hurt the deep ITM call.

Speed comes from structure, not language. The 15–20× gain from leaving the interpreter and the 7× gain from eight cores were the real speed-ups; rewriting the same scalar loop in C++ gained nothing over NumPy. The most useful C++ property was not raw speed but control: a constant-memory design that kept cost flat at large N, and a random-number design that made the multithreaded result bitwise reproducible.

9. Limitations

The main limitation is that every option tested has an exact Black-Scholes price; real uses of Monte Carlo are for products without one, where coverage cannot be checked directly.

- **Only European options under Black-Scholes.** Constant volatility, no dividends and no path dependence. Methods may rank differently for path-dependent options or models with jumps.
- **One importance-sampling design.** I used a simple drift shift; its size was not optimised, and it was tested mainly on far out-of-the-money calls.
- **Coverage resolution.** With 500 seeds, a single coverage figure is only accurate to about ±0.02. All 16 grid options shared one seed set, so an unlucky set moves them together: antithetic coverage first read about 0.92 across the grid, but 5,000 seeds gave 0.944–0.948.
- **The paying-path threshold is empirical.** "About 50 paying paths" comes from one sweep of six-month calls; I did not test it on other payoffs.
- **The C++ comparison is not like-for-like.** Scalar C++ was compared with vectorised NumPy; a hand-vectorised C++ core was not written. The 8.2× cost advantage compares 24 threads with one; NumPy split across several processes would close part of that gap.
- **One machine and operating system.** Per-path costs, cache effects and thread scaling depend on the hardware.
- **The control coefficient.** The default fits it on the same paths, which is biased below about N = 10,000; small-N results should use a pilot sample.

10. Conclusion and future work

Numerical and variance-reduction techniques improved the accuracy of Monte Carlo option pricing substantially, but only for options whose important outcomes the simulation already sampled. Plain Monte Carlo converged at the theoretical $1/\sqrt{N}$ rate with an honest error bar, until fewer than about 50 paths paid anything. Antithetic and control variates reduced variance by up to 521× on ordinary options, but left that failure in place and, in one case, created it. Importance sampling was the only method that restored correct 95% coverage on rare-event options, because it was the only one that changed where the paths went.

On efficiency, the largest gains came from leaving the Python interpreter (15–20×) and from parallelism (8.2× lower cost at 10^7 paths), not from rewriting the same loop in C++. The broader lesson is that a pricing method should be judged by whether its error bar can be trusted, not only by how small that error bar is.

Future work:

1. Test the methods on path-dependent options (Asian, barrier) and on a model with jumps, where no exact price exists and coverage must be checked against a very long reference run.
2. Choose the importance-sampling shift optimally and test it across the full option grid.
3. Find whether "about 50 paying paths" holds for other payoffs and maturities, and whether it can be turned into an automatic warning.
4. Write a hand-vectorised C++ core and a multi-process NumPy version, for a like-for-like speed comparison.